

Module 3

IEEE 1364-2005 (Verilog-2005)

Learning objectives

- Master the final Verilog-only standard (IEEE 1364-2005), its clarifications over 2001, and the synt...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["IEEE 1364-2005 Context"]

T2["defparam Deprecation (2005)"]

T1 --> T2

T3["Synthesizable Subset (1364-2)"]

T2 --> T3

Master the final Verilog-only standard (IEEE 1364-2005), its clarifications over 2001, and the synthesizable subset as a stable baseline...

Overview

- This module covers IEEE Std 1364-2005 (Verilog-2005), the last standard that defines Verilog alone...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 3.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module3/``.
- Work through labs in order — Open ``module3/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module3.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module3/CHECKLIST.md`` before starting the next module.

Design architecture

1. Repository layout (module3/)

- 1364-2005 clarifications and deprecations over 2001
- Focus: avoid defparam, use localparam, understand 2005 vs 2001 deltas
- module3/examples/ — migration and best-practice labs
- scripts/module3.sh — run all 2005-focused examples

```
Rtl Architecture
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph DUT["module3/dut"]
CNT["counter_2005"]
FSM["fsm_2005 case FSM"]
PC["param_chain hierarchy"]
end
```

2. Parameter and constant flow

- localparam — named constants
inside module, not overridable
- Parameter override at
instantiation `#(.WIDTH(n))`
preferred over `defparam`
- 2005 standard marks `defparam` as
deprecated

```
Verification Architecture
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
SIM["Icarus sim"] --> CNT
SIM --> FSM
SIM --> PC
FSM --> COV["state transition vectors"]
CNT --> WRAP["wrap at MAX check"]
```

3. Design intent unchanged from 2001

- Same ANSI, @*, generate, signed features as Module 2
- 2005 adds LRM clarifications — behavior you already use stays valid
- Migration labs show 2001 → 2005 style tweaks only

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TB
  M3["M3[\"module3.sh\"]"] --> TC["TC[\"test_counter_2005\"]"]
  M3 --> TF["TF[\"test_fsm_2005\"]"]
  M3 --> TP["TP[\"test_param_chain\"]"]
  TC --> SYNTH["SYNTH[\"synthesizable subset OK\"]"]
```

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph DUT["module3/dut"]

CNT["counter_2005"]

FSM["fsm_2005 case FSM"]

PC["param_chain hierarchy"]

end

Module 3: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

SIM["Icarus sim"] --> CNT

SIM --> FSM

SIM --> PC

FSM --> COV["state transition vectors"]

CNT --> WRAP["wrap at MAX check"]

Module 3: stimulus, observation, and checking.

Parameter override at instantiation

```
/*
 * Parameter override at instantiation - IEEE 1364-2005
 * No defparam (deprecated); use #(.PARAM(value)) at instantiation
 * only.
 */

module adder #(parameter WIDTH = 8) (
    input  wire [WIDTH-1:0] a,
    input  wire [WIDTH-1:0] b,
    output wire [WIDTH-1:0] y
);
    assign y = a + b;
endmodule

module top;
    reg [3:0] a4, b4;
    reg [7:0] a8, b8;
    #
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module3/examples/<example>/ — run make run from that directory

Key files to study

Open these in the repo

- module3/examples/no_defparam/ — replace defparam with #() override
- module3/examples/parameters/ — parameter override at instantiation
- scripts/module3.sh

Verification & testing methods

1. Execution and regression

- make run per example;
./scripts/module3.sh for full
sweep
- Watch for defparam warnings when
running deprecated-style labs

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M3["module3.sh"] --> TC["test_counter_2005"]

M3 --> TF["test_fsm_2005"]

M3 --> TP["test_param_chain"]

TC --> SYNTH["synthesizable subset OK"]

2. Migration checks

- Replace defparam with #()
 override at instantiation
- Confirm localparam names match
 across parent and child

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M3["module3.sh"] --> TC["test_counter_2005"]

M3 --> TF["test_fsm_2005"]

M3 --> TP["test_param_chain"]

TC --> SYNTH["synthesizable subset OK"]

Avoid defparam — use #() instead

```
/*  
 * No defparam - IEEE 1364-2005  
 * defparam is deprecated; override only at instantiation.  
 * This example shows correct hierarchy: parent passes parameter to  
child at inst.  
 */  
  
module child #(parameter W = 4) (  
    input  wire [W-1:0] a,  
    output wire [W-1:0] y  
);  
    assign y = a;  
endmodule  
  
module parent #(parameter W = 4) (  
    input  wire [W-1:0] a,  
    output wire [W-1:0] y  
    //
```

Syllabus topics

Protocol & design details

Detail: What 1364-2005 Is

- Final Verilog-only standard: No SystemVerilog (no ``logic``, interfaces, packages, classes).
- Clarifications: Refined wording and examples for 2001 features (e.g. generate, signed, scheduling).
- Minor additions: Additional system tasks and small syntax clarifications.
- Stable baseline: Many synthesis and style guides treat 1364-2005 as the Verilog baseline.

Detail: What 1364-2005 Adds or Clarifies Over 2001 (1)

- Clarifications: Scheduling, blocking vs nonblocking semantics, and generate behavior.
- System tasks: Additional or clarified tasks (e.g. ``$realtime``, file I/O behavior).
- Deprecations: e.g. `defparam` is deprecated (use instantiation parameter override instead).
- Attributes: Clarified syntax and usage for tool-specific attributes.

Detail: What 1364-2005 Adds or Clarifies Over 2001 (2)

- No major new language features: The big step to new features is 1800 (Module 4).

Detail: What 1364-2005 Still Does Not Include

- No SystemVerilog: no ``logic``,
 ``always_comb``/``always_ff``, interfaces, packages,
 classes, assertions...
- No interfaces or type parameters (those are 1800).

Detail: Why defparam Is Deprecated

- Problem: Overrides a parameter deep in hierarchy from outside; hard to trace and tool-dependent.
- Preferred: Override at instantiation: ``sub
#(.WIDTH(16)) u_sub (...);``

Detail: Correct Practice: Parameter Override at Instantiation

- Recommendation: Do not use defparam in new RTL; use named parameter override in the instantiation.

Detail: Generally Synthesizable (RTL Subset) (1)

- Modules: ANSI ports, parameter, localparam.
- Nets and variables: wire, reg (with correct usage).
- assign: Combinational logic; single driver per net.
- always: Combinational with ``always @*``; sequential with ``always @(posedge clk)`` (or `negedge`) and no...

Detail: Generally Synthesizable (RTL Subset) (2)

- generate: generate for, generate if/else for structure.
- Operators: Arithmetic, logical, bitwise, relational, shift, concatenation, conditional (``? :``).
- case / casez / casex: With care (full case, parallel case, or tool directives).
- Tasks and functions: Without delays; used for combinational or single-cycle helpers.

Detail: Generally Not Synthesizable (or Tool-Specific) (1)

- Delays: ``#n`` in RTL (used for testbenches only).
- initial: Except for initialization of reg/mem in FPGAs (tool-dependent).
- Event controls: ``@(posedge clk)`` is fine; complex event lists may not map.
- Fork/join: Usually testbench-only.

Detail: Generally Not Synthesizable (or Tool-Specific) **(2)**

- System tasks: e.g. ``$display``, ``$finish`` — testbench/simulation only.
- Pure latch inference: Unintended latches from incomplete case/if; use full case or default.

Detail: Best Practices for Synthesis (1364-2005)

- Combinational: Use ``always @*`` and blocking ``=``; assign all outputs in all branches (or default) to...
- Sequential: Use ``always @(posedge clk)`` (and optionally ``negedge rst_n``) and nonblocking ``<=```; do n...
- One driver per net: No multiple assign to the same wire; no multiple always driving the same reg in...
- Avoid latches: In combinational always, cover all cases or use a default assignment.

Detail: Rules of Thumb

- Combinational always (``always @*``): Use blocking `=`. Order of statements can matter (same simulation results may vary).
- Sequential always (``always @(posedge clk)``): Use nonblocking `<=`. Order does not matter within the block.
- Do not mix blocking and nonblocking for the same variable in the same always block.

Detail: full_case (Tool-Dependent)

- Intent: “Every possible case value is covered; no latch.”
- Syntax: Often an attribute, e.g. ``(* full_case *)`` or tool-specific comment.
- Use: When the designer guarantees all cases are covered (e.g. one-hot state encoding).

Detail: parallel_case (Tool-Dependent)

- Intent: “At most one case arm matches; can be implemented as parallel mux.”
- Syntax: Often an attribute, e.g. ``(* parallel_case *)`` or tool-specific.
- Use: When at most one branch is true (e.g. one-hot); misuse can hide bugs.

Verification flow (reference)

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M3["module3.sh"] --> TC["test_counter_2005"]

M3 --> TF["test_fsm_2005"]

M3 --> TP["test_param_chain"]

TC --> SYNTH["synthesizable subset OK"]

Stimulus, DUT response, and check points for this module.

1. IEEE 1364-2005 Context (1/2)

- Final Verilog-only standard: No SystemVerilog (no ``logic``, interfaces, packages, classes).
- Clarifications: Refined wording and examples for 2001 features (e.g. generate, signed, scheduling).
- Minor additions: Additional system tasks and small syntax clarifications.
- Stable baseline: Many synthesis and style guides treat 1364-2005 as the Verilog baseline.
- Clarifications: Scheduling, blocking vs nonblocking semantics, and generate behavior.
- System tasks: Additional or clarified tasks (e.g. ``$realtime``, file I/O behavior).

1. IEEE 1364-2005 Context (2/2)

- Deprecations: e.g. `defparam` is deprecated (use instantiation parameter override instead).
- Attributes: Clarified syntax and usage for tool-specific attributes.
- No major new language features: The big step to new features is 1800 (Module 4).
- No SystemVerilog: no ``logic``, ``always_comb``/``always_ff``, interfaces, packages, classes, assertions...
- No interfaces or type parameters (those are 1800).

2. defparam Deprecation (2005)

- Problem: Overrides a parameter deep in hierarchy from outside; hard to trace and tool-dependent.
- Preferred: Override at instantiation: ``sub
#(.WIDTH(16)) u_sub (...);``
- Recommendation: Do not use defparam in new RTL; use named parameter override in the instantiation.
- Why defparam Is Deprecated
- Correct Practice: Parameter Override at Instantiation

3. Synthesizable Subset (1364-2005) (1/4)

- Modules: ANSI ports, parameter, localparam.
- Nets and variables: wire, reg (with correct usage).
- assign: Combinational logic; single driver per net.
- always: Combinational with ``always @*``; sequential with ``always @(posedge clk)`` (or `negedge`) and no...
- generate: generate for, generate if/else for structure.
- Operators: Arithmetic, logical, bitwise, relational, shift, concatenation, conditional (``? :``).

3. Synthesizable Subset (1364-2005) (2/4)

- case / casez / casex: With care (full case, parallel case, or tool directives).
- Tasks and functions: Without delays; used for combinational or single-cycle helpers.
- Signed: signed types and \$signed/\$unsigned as supported by the tool.
- Delays: `#n` in RTL (used for testbenches only).
- initial: Except for initialization of reg/mem in FPGAs (tool-dependent).
- Event controls: `@(posedge clk)` is fine; complex event lists may not map.

3. Synthesizable Subset (1364-2005) (3/4)

- Fork/join: Usually testbench-only.
- System tasks: e.g. ``$display``, ``$finish`` — testbench/simulation only.
- Pure latch inference: Unintended latches from incomplete case/if; use full case or default.
- Combinational: Use ``always @*`` and blocking ``=``; assign all outputs in all branches (or default) to...
- Sequential: Use ``always @(posedge clk)`` (and optionally ``negedge rst_n``) and nonblocking ``<=```; do n...
- One driver per net: No multiple assign to the same wire; no multiple always driving the same reg in...

3. Synthesizable Subset (1364-2005) (4/4)

- Avoid latches: In combinational always, cover all cases or use a default assignment.

4. Blocking vs Nonblocking (Clarified in 2005)

- Combinational always (``always @*``): Use blocking `=`. Order of statements can matter (same simulation, different results).
- Sequential always (``always @(posedge clk)``): Use nonblocking `<=`. Order does not matter within the block.
- Do not mix blocking and nonblocking for the same variable in the same always block.
- Rules of Thumb

5. Full Case and Parallel Case (Synthesis Hints)

- Intent: “Every possible case value is covered; no latch.”
- Syntax: Often an attribute, e.g. ``(* full_case *)`` or tool-specific comment.
- Use: When the designer guarantees all cases are covered (e.g. one-hot state encoding).
- Intent: “At most one case arm matches; can be implemented as parallel mux.”
- Syntax: Often an attribute, e.g. ``(* parallel_case *)`` or tool-specific.
- Use: When at most one branch is true (e.g. one-hot); misuse can hide bugs.

6. Summary: 1364 Evolution (1995 → 2001 → 2005)

- See docs/MODULE3.md

Hands-on examples

Module 3 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module3_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 1: No defparam; override at instantiation only

- defparam deprecated; use ``#(.PARAM(value))`` at instantiation

Demo: No defparam; override at instantiation only

```
$ cd module3/examples/parameters && make clean && make run
```

Terminal: ex01_parameters

```
rm -f top
iverilog -o top param_override.v
vvp top
Parameter override at instantiation (1364-2005): no defparam
 4-bit: 3 + 5 = 8
 8-bit: 10 + 20 = 30
param_override.v:29: $finish called at 20 (1s)
```

Example 2: Parameter passed through hierarchy (parent → child) at ins...

- No defparam anywhere; override only at inst

Demo: Parameter passed through hierarchy (parent → child) at instanti...

```
$ cd module3/examples/no_defparam && make clean && make run
```

```
Terminal: ex02_no_defparam  
rm -f top  
iverilog -o top no_defparam.v  
vvp top  
No defparam (1364-2005): parameter only at instantiation  
  a=ab y=ab  
no_defparam.v:31: $finish called at 10 (1s)
```


Example 3: Combinational (always @*, blocking) and sequential (posedg...

- One driver per net; avoid latches; no delays in RTL

Demo: Combinational (always @*, blocking) and sequential (posedge clk...

```
$ cd module3/examples/synthesizable && make clean && make run
```

Terminal: ex03_synthesizable

```
rm -f top
iverilog -o top comb_seq_style.v
vvp top
Synthesizable style (1364-2005): combo @* blocking, seq posedge nonblocking
y_comb=1 q_seq=0
y_comb=1 q_seq=1
y_comb=1 q_seq=1
comb_seq_style.v:46: $finish called at 45 (1s)
```

Example 4: Single assign to wire; single always driving reg

- No multiple drivers per net

Demo: Single assign to wire; single always driving reg

```
$ cd module3/examples/one_driver && make clean && make run
```

Terminal: ex04_one_driver

```
rm -f top
iverilog -o top one_driver.v
vvp top
One driver per net (1364-2005): single assign, single always per output
y_assign=1 q_reg=0
y_assign=1 q_reg=1
y_assign=1 q_reg=1
one_driver.v:41: $finish called at 45 (1s)
```

Example 5: Side-by-side combinational vs sequential blocks with corre...

- = in combinational; <= in sequential; do not mix for same variable

Demo: Side-by-side combinational vs sequential blocks with correct as...

```
$ cd module3/examples/procedural && make clean && make run
```

```
Terminal: ex05_procedural
rm -f top
iverilog -o top blocking_nonblocking.v
vvp top
Blocking vs nonblocking (1364-2005): = in combo, <= in seq
  combo: y=1 z=1
  seq: q1=1 q2=x (q2 gets old q1)
  seq: q1=1 q2=1 (q2 gets old q1)
  seq: q1=1 q2=1 (q2 gets old q1)
blocking_nonblocking.v:49: $finish called at 35 (1s)
```

Example 6: Two-stage pipeline: $q1 \leq d$; $q2 \leq q1$ (nonblocking only)

- q2 gets old q1 (previous cycle); sequential block only

Demo: Two-stage pipeline: $q1 \leq d$; $q2 \leq q1$ (nonblocking only)

```
$ cd module3/examples/pipeline && make clean && make run
```

Terminal: ex06_pipeline

```
rm -f top
iverilog -o top pipeline.v
vvp top
Two-stage pipeline (1364-2005): nonblocking only; q2 <= q1 (old value)
d= 1 q1= 0 q2= 0
d= 2 q1= 2 q2= 0
d= 3 q1= 2 q2= 2
pipeline.v:42: $finish called at 45 (1s)
```


Example 7: full_case / parallel_case attributes (or tool pragmas) whe...

- Synthesis hints only; 1364-2005 has no unique/priority case

Demo: full_case / parallel_case attributes (or tool pragmas) where su...

```
$ cd module3/examples/case_styles && make clean && make run
```

Terminal: ex07_case_styles

```
rm -f top
iverilog -o top case_full_parallel.v
vvp top
Case full coverage (1364-2005): default to avoid latch
  sel=0 y=1
  sel=1 y=2
  sel=2 y=4
  sel=3 y=8
case_full_parallel.v:42: $finish called at 40 (1s)
```

Example 8: Default before case; all paths assign output

- No latch from incomplete case/if

Demo: Default before case; all paths assign output

```
$ cd module3/examples/avoid_latch && make clean && make run
```

Terminal: ex08_avoid_latch

```
rm -f top
iverilog -o top avoid_latch.v
vvp top
Avoid latch (1364-2005): default + case so all paths assign y
  sel=0 y=1
  sel=2 y=4
  sel=3 y=8
avoid_latch.v:40: $finish called at 30 (1s)
```

Example 9: Function without delays used in always @* (e.g. min)

- Combinational helper; no delays in RTL

Demo: Function without delays used in always @* (e.g. min)

```
$ cd module3/examples/function_synth && make clean && make run
```

Terminal: ex09_function_synth

```
rm -f top
iverilog -o top function_synth.v
vvp top
Synthesizable function (1364-2005): no delays; used in always @*
  min( 50, 30)= 30
  min( 10, 99)= 10
function_synth.v:29: $finish called at 20 (1s)
```

Example 10: Small design using only 1364-2005 features (no 1800)

- Checklist for “pure Verilog” RTL

Demo: Small design using only 1364-2005 features (no 1800)

```
$ cd module3/examples/summary && make clean && make run
```

Terminal: ex10_summary

```
rm -f top
iverilog -o top summary_2005.v
vvp top
1364-2005 summary: ANSI, param at inst, nonblocking, no defparam
  a= 3 b= 5 sum= 0
  a= 3 b= 5 sum= 8
  a= 3 b= 5 sum= 8
summary_2005.v:37: $finish called at 45 (1s)
```


Common pitfalls

Using defparam in New Code

- Mistake: Overriding parameters with defparam for convenience.
- Reality: defparam is deprecated in 1364-2005; tools may warn or restrict.
- Correct: Override at instantiation: ``sub
#(.WIDTH(16)) u_sub (...);``
- Why: Clear, hierarchical parameter flow; better for synthesis and maintenance.

Mixing Blocking and Nonblocking for Same Variable

- Mistake: Using both ``=`` and ``<=` for the same reg in one always block.
- Reality: Leads to simulation/synthesis mismatch and race conditions.
- Correct: Use only ``=`` in combinational always; only ``<=` in sequential always for that variable.
- Why: 1364-2005 semantics assume consistent use; mixing is undefined in practice.

Assuming unique/priority case in 1364-2005

- Mistake: Writing ``unique case`` or ``priority case``
(SystemVerilog) and expecting 1364-2005 tools to...
- Reality: Those keywords are in IEEE 1800, not 1364.
- Correct: Use `full_case`/`parallel_case` attributes if
the tool supports them; otherwise ensure full co...
- Why: Avoids tool errors and clarifies design intent.

Latches from Incomplete case/if

- Mistake: Combinational always with case or if that does not assign in all paths.
- Reality: Synthesis infers a latch; simulation may hold previous value.
- Correct: Assign default before case, or cover all cases; avoid incomplete if/else.
- Why: Latches are rarely intended in synchronous RTL; explicit defaults avoid them.

Using SystemVerilog in “1364-2005” Project

- Mistake: Using ``logic``, ``always_comb``, or interfaces while targeting “Verilog only.”
- Reality: Those are 1800 (SystemVerilog); 1364-2005 does not define them.
- Correct: Use `wire/reg` and `always @*` / `always @(posedge clk)` for strict 1364-2005.
- Why: Keeps tool and project consistently on one standard.

Practice & assessment

What you should know

- ✓ Describe what 1364-2005 adds or clarifies over 1364-2001 (no major new constructs)
- ✓ Avoid defparam and use instantiation parameter override only
- ✓ Apply the synthesizable subset of 1364-2005 (combinational vs sequential, one driver, no latches)
- ✓ Use blocking assignment in combinational always and nonblocking in sequential always
- ✓ Use tool-supported full_case/parallel_case (or equivalent) where appropriate
- ✓ Summarize 1364 evolution (1995 → 2001 → 2005) and when to stay Verilog-only vs adopt 1800

Exercises

- Remove defparam
- Synthesizable Checklist
- Blocking vs Nonblocking
- Case and Latches
- 1364-2005 Summary

Exercises

- Using defparam in New Code
- Mixing Blocking and Nonblocking for Same Variable
- Assuming unique/priority case in 1364-2005
- Latches from Incomplete case/if
- Using SystemVerilog in “1364-2005” Project

Summary & next steps

- Master the final Verilog-only standard (IEEE 1364-2005), its clarifications over 2001, and the synt...
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: Next module in course